Contents

1 Basic 1
1.1 default 1
1.2 奇妙的東西 1
1.3 random 1

2 Math 1
2.1 basic 1
2.2 快速冪 1
2.3 大數運算 1
2.4 極座標排序 2

3 Binary Search 2
3.1 二分搜 2
3.2 三分搜 2

4 DataStructure 2
4.1 DSU(並查集) 2
4.2 FenwickTree/BIT 3
4.3 Seg/線段樹 3

5 Graph 3
5.1 Topo/拓樸 3
5.2 ola/歐拉 3

6 Math2 4
6.1 Theorem 4
6.2 Prime 4

1 Basic

1.1 default

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define idonthavegirlfriend ios::sync_with_stdio(0),cin
    .tie(0);
#define pb push_back
#define endl "\n"
#define all(v) (v).begin(),(v).end()
void solve(){

}
signed main(){
    idonthavegirlfriend
    int _=1;
    //cin>>_;
    while(_--){
        solve();
    }
}
```

1.2 奇妙的東西

```
priority_queue<T> //由大到小
priority_queue<T,vector<T>,greater<T>> //由小到大
__builtin_popcountll// 二進位有幾個1(記得這是long long)
__builtin_clzll          // 左起第一個1之前0的個數
__builtin_parityll       // 1的個數的奇偶性
__builtin_mul_overflow(a,b,&h) // a*b是否溢位,h = a * b
    ;
__builtin_add_overflow(a,b,&h)
```

1.3 random

```
mt19937 mt(chrono::steady_clock::now().time_since_epoch
    ().count());
//mt19937_64 mt() -> return randnum
int randint(int l, int r){
    uniform_int_distribution<> dis(l, r); return dis(mt
        );
}
```

2 Math

2.1 basic

```
(a,b) 有包含
[a,b] 沒包含
```

2.2 快速冪

```
int ksm(int x, int y) {
    int ans = 1;
    while (y) {
        if (y & 1)
            ans = ans * x;
        x = x * x;
        y >>= 1;
    }
    return ans;
}

// 矩陣乘法
vector<vector<int>> mul(const vector<vector<int>> &A,
                        const vector<vector<int>> &B) {
    int n = A.size();
    vector<vector<int>> C(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) if (A[i][k]) {
            for (int j = 0; j < n; j++) {
                C[i][j] = (C[i][j] + A[i][k] * B[k][j])
                    % MOD;
            }
        }
    }
    return C;
}

// 矩陣快速冪
vector<vector<int>> ksm(vector<vector<int>> base, int
    exp) {
    int n = base.size();
    vector<vector<int>> res(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) res[i][i] = 1;

    while (exp > 0) {
        if (exp & 1) res = mul(res, base);
        base = mul(base, base);
        exp >>= 1;
    }
    return res;
}
```

2.3 大數運算

```
//加法
string add(string num1, string num2) {
    string res = "";
    int i = num1.length() - 1, j = num2.length() - 1,
        carry = 0;
    while (i >= 0 || j >= 0 || carry) {
        int sum = carry + (i >= 0 ? num1[i--] - '0' :
            0) + (j >= 0 ? num2[j--] - '0' : 0);
        carry = sum / 10;
        res += to_string(sum % 10);
    }
    reverse(res.begin(), res.end());
    return res;
}

//減法
string sub(string num1, string num2) {
    string res = "";
    int i = num1.length() - 1, j = num2.length() - 1,
        borrow = 0;
    while (i >= 0) {
        int sub = (num1[i--] - '0') - (j >= 0 ? num2[j
            --] - '0' : 0) - borrow;
        if (sub < 0) {
            sub += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        res += to_string(sub);
    }
    while (res.length() > 1 && res.back() == '0') res.
        pop_back(); // 去除前導0
    reverse(res.begin(), res.end());
```

```
    return res;
}

//乘法
string mul(string num1, string num2) {
    int n = num1.size(), m = num2.size();
    vector<int> res(n + m, 0);

    for (int i = n - 1; i >= 0; i--) {
        for (int j = m - 1; j >= 0; j--) {
            int p = (num1[i] - '0') * (num2[j] - '0');
            int sum = p + res[i + j + 1];

            res[i + j + 1] = sum % 10;
            res[i + j] += sum / 10;
        }
    }
    string ans = "";
    for (int x : res) {
        if (!(ans.empty() && x == 0))  // 跳過前導0
            ans += (char)(x + '0');
    }
    return ans.empty() ? "0" : ans;
}
//除法
bool smaller(string a, string b) {
    if (a.size() != b.size()) return a.size() < b.size
        ();
    return a < b;
}
string div(string num1, string num2) {
    string cur = "", ans = "";
    for (char c : num1) {
        cur += c;
        // 去掉前導0
        while (cur.size() > 1 && cur[0] == '0') cur.
            erase(cur.begin());

        int x = 0;
        while (!smaller(cur, num2)) {
            cur = sub(cur, num2);
            x++;
        }
        ans += (char)(x + '0');
    }
    // 去掉前導0
    while (ans.size() > 1 && ans[0] == '0') ans.erase(
        ans.begin());
    return ans;
}
string mod(string num1, string num2) {//餘數
    string cur = "";

    for (char c : num1) {
        cur += c;
        while (cur.size() > 1 && cur[0] == '0') cur.
            erase(cur.begin());
        while (!smaller(cur, num2)) {
            cur = sub(cur, num2);
        }
    }
    return cur;
}
```

2.4 極座標排序

```
//由+x軸往逆時針排序
struct point {
    int x, y;
};
long long cross(const point &a, const point &b) {
    return (long long) a.x * b.y - (long long) a.y * b.
        x;
}
bool cmp(const point &a, const point &b) {
    int ah = (a.y < 0 or (a.y == 0 and a.x < 0));
    int bh = (b.y < 0 or (b.y == 0 and b.x < 0));
    if (ah != bh) return ah < bh;
    return cross(a, b) > 0;
}
void argument_sort(vector<point> &points) {
    sort(points.begin(), points.end(), cmp);
}
```

3 Binary Search

3.1 二分搜

```
int bsearch_1(int l, int r)
{
    while (l < r)
    {
        int mid = l + r >> 1;
        if (check(mid)) r = mid;
        else l = mid + 1;
    }
    return l;
}
// ................oooooooooo

int bsearch_2(int l, int r)
{
    while (l < r)
    {
        int mid = l + r + 1 >> 1;
        if (check(mid)) l = mid;
        else r = mid - 1;
    }
    return l;
}
// oooooooooo..................

int m = *ranges::partition_point(views::iota(0LL,(int)1
    e9+9),[&](int a){
    return check(a) > k;
    });
//[begin,last)
//111111100000000000
//搜左邊數過來第一個 0
//都是 1 會回傳 last

int partitionpoint(int L,int R,function<bool(int)> chk)
    {
  int l = L,r = R-1;
  while(r - l > 10){
    int m = l + (r-l)/2;
    if(chk(m)) l = m;
    else r = m;
  }
    int m = l;
    while(m <= r){
        if(!chk(m)) break;
        ++m;
    }
  if(!chk(m)) return m;
  else return R;
}

//手工
```

3.2 三分搜

```
int l = 1,r = 100;
while(l < r) {
    int lmid = l + (r - l) / 3; // l + 1/3區間大小
    int rmid = r - (r - l) / 3;  // r - 1/3區間大小
    lans = cal(lmid),rans = cal(rmid);
    // 求凹函數極小直
    if(lans <= rans) r = rmid - 1;
    else l = lmid + 1;
}
```

4 DataStructure

4.1 DSU(並查集)

```
struct DSU {
    vector<int> f, sz;
    int cnt;
    DSU(int n) : f(n), sz(n) {
        for (int i = 0; i < n; i++) {
            f[i] = i;
```

```
            sz[i] = 1;
        }
    }
    int find(int x) {
        if (x == f[x]) return x;
        return f[x] = find(f[x]);
    }
    void merge(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return;
        if (sz[x] < sz[y])
            swap(x, y);
        f[y] = x;
        sz[x] += sz[y];
        cnt--;
    }
    bool same(int a, int b) {
        return find(a) == find(b);
    }
    int groups() {//找有幾群
        return cnt;
    }
};
```

4.2 FenwickTree/BIT

```
struct Fenwick {//這個是1base的
    int n;
    vector<int> s;
    int lowbit(int x) { return x & -x; }
    Fenwick(int _n) {
        n = _n + 1;
        s.clear(), s.resize(n, 0);
    }
    void update(int i, int v) {  // 更新數值
        for (; i < n; i += lowbit(i))
            s[i] += v;
    }
    int query(int x) {  // 查詢前綴和(有包含)
        int pre = 0;
        for (; x; x -= lowbit(x))
            pre += s[x];
        return pre;
    }
    Fenwick(vector<int> a) {// 建構資料結構
        n = a.size(); // a 是 0-based
        s.clear(), s.resize(n + 1, 0); // s 是 1-based
        for (int i = 1; i <= n; i++)
            update(i, a[i - 1]);
    }
};
//使用:
Fenwick fw(a);//a is vector
Fenwick fw(n);//n is int
```

4.3 Seg/線段樹

```
struct Seg{
    #define cl(x) (x<<1)
    #define cr(x) (x<<1)+1
    int _n;
    vector<int> f;
    vector<int> arr;
    int base;
    Seg(int n):_n(n),f(4*n){}
    Seg(vector<int> a,int b){//
        _n=a.size();
        f.resize(4*_n+1,0);
        arr.resize(_n);
        for(int i=0;i<_n;i++){
            arr[i]=a[i];
        }
        base=b;
        build(1,base,_n-1+base);

    }
    void build(int id,int l,int r){
        if(l==r){
            f[id]=arr[l];
            return ;
        }
        int mid=(l+r)>>1;
        build(cl(id),l,mid);
        build(cr(id),mid+1,r);
        pull(id);
    }
    void pull(int id){
        f[id]=min(f[cl(id)],f[cr(id)]);
    }
    void update(int id,int l,int r,int x,int v){
        if(l==r){
            f[id]=v;
            return;
        }
        int mid=(l+r)>>1;
        if(x<=mid) update(cl(id),l,mid,x,v);
        if(mid<x) update(cr(id),mid+1,r,x,v);
        pull(id);
    }
    int query(int id,int l,int r,int sl,int sr){
        if(sl<=l and sr>=r){
            return f[id];
        }
        int mid=(l+r)>>1;
        int res=1e18;
        int ll=1e18;int rr=1e18;
        if(sl<=mid){
            ll=query(cl(id),l,mid,sl,sr);
        }
        if(sr>mid){
            rr=query(cr(id),mid+1,r,sl,sr);
        }
        res=min(ll,rr);
        return res;
    }
};
```

5 Graph

5.1 Topo/拓樸

```
vector<int> edge[MAXN], ans;
int deg[MAXN];

void topo(int n) {
    queue<int> q;

    for (int i = 1; i <= n; i++)
        if (!deg[i])
            q.push(i);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ans.push_back(u);

        for (int v : edge[u]) {
            deg[v]--;
            if (!deg[v])
                q.push(v);
        }
    }
}
```

5.2 ola/歐拉

```
//歐拉路徑(無向圖)兩個奇數點或者全偶點
//歐拉路徑(有向圖)條件:起點out=in+1,終點in=out+1,其他點
    in=out
//歐拉迴路(無向圖)所有點偶數點
//歐拉迴路(有向圖)所有點in=out
vector<int> anse;//邊
vector<int> ansp;//點
vector<bool> vis(m,false);
function<void(int)> dfs=[&](int u){
    while(!mp[u].empty()){
        auto [x,e]=mp[u].back();
        mp[u].pop_back();
        if(vis[e]){
            continue;
        }
        vis[e]=true;
```

```
        dfs(x);
        anse.pb(e);
    }
    ansp.pb(u);
};
```

6 Math2

6.1 Theorem

- Lucas's Theorem :
  For $n, m \in \mathbb{Z}^*$ and prime P, $C(m,n)$ mod $P = \Pi(C(m_i, n_i))$ where m_i is the i-th digit of m in base P.
- Stirling approximation :
  $n! \approx \sqrt{2\pi n}(\frac{n}{e})^n e^{\frac{1}{12n}}$
- Stirling Numbers(permutation $|P| = n$ with k cycles):
  $S(n,k) =$ coefficient of x^k in $\Pi_{i=0}^{n-1}(x+i)$
- Stirling Numbers(Partition n elements into k non-empty set):
  $S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$
- Pick's Theorem : $A = i + b/2 - 1$
  A: Area、i: grid number in the inner、b: grid number on the side
- Catalan number : $C_n = \binom{2n}{n}/(n+1)$
  $C_n^{n+m} - C_{n+1}^{n+m} = (m+n)!\frac{n-m+1}{n+1} \quad for \quad n \geq m$
  $C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$
  $C_0 = 1 \quad and \quad C_{n+1} = 2(\frac{2n+1}{n+2})C_n$
  $C_0 = 1 \quad and \quad C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i} \quad for \quad n \geq 0$
- Euler Characteristic:
  planar graph: $V - E + F - C = 1$
  convex polyhedron: $V - E + F = 2$
  V, E, F, C: number of vertices, edges, faces(regions), and components
- Kirchhoff's theorem :
  $A_{ii} = deg(i), A_{ij} = (i,j) \in E \ ? -1 : 0$, Deleting any one row, one column, and cal the det(A)
- Polya' theorem (c is number of color，m is the number of cycle size):
  $(\sum_{i=1}^{m} c^{gcd(i,m)})/m$
- Burnside lemma:
  $|X/G| = \frac{1}{|G|}\sum_{g \in G}|X^g|$
- 錯排公式: (n 個人中，每個人皆不再原來位置的組合數):
  $dp[0] = 1; dp[1] = 0;$
  $dp[i] = (i-1) * (dp[i-1] + dp[i-2]);$
- Bell 數 (有 n 個人, 把他們拆組的方法總數) :
  $B_0 = 1$
  $B_n = \sum_{k=0}^{n} s(n,k) \quad (second - stirling)$
  $B_{n+1} = \sum_{k=0}^{n}\binom{n}{k}B_k$
- Wilson's theorem :
  $(p-1)! \equiv -1 (mod \ p)$
- Fermat's little theorem :
  $a^p \equiv a (mod \ p)$
- Euler's totient function:
  $A^{B^C} \ mod \ p = pow(A, pow(B, C, p-1)) mod \ p$
- 歐拉函數降冪公式:
  A^B mod $C = A^{B \text{ mod } \phi(c) + \phi(c)}$ mod C
- 6 的倍數:
  $(a-1)^3 + (a+1)^3 + (-a)^3 + (-a)^3 = 6a$
- Standard young tableau (標準楊表):
  $\lambda = (\lambda_1 \geq \cdots \geq \lambda_k)$,$\sum \lambda_i = n$ denoted by $\lambda \vdash n$
  $\lambda \vdash n$ 意思為 λ 整數拆分 n eg. $n = 10, \lambda = (6,4)$ 此拆分可表示一種楊表形狀。
  楊表: 第 1 列 λ_1 行 $\cdots$ 第 k 列 λ_k 行的方格圖。
  標準楊表: 每列從左到右遞增，每行從上到下遞增。
  Let T 為某一 Permutation 跑 RSK 後的標準楊表，則此 Permutation 的 LDS、LIS 長度分別為 T 的列、行數。
- RSK Correspondence:
  A permutation is bijective to (P, Q) 一對標準楊表
  P : Permutation 跑 RSK 算法的結果，可為半標準楊表。
  Q : 可用來還原 Permutation (像排列矩陣)。
- Hook length formula (形狀為 λ 的標準楊表個數):
  $f^\lambda = \frac{n!}{\prod h_\lambda(i,j)}$
  $h_\lambda(i,j) =$ number of pair (x,y) where $(x = i \vee y = j) \wedge (x,y) \geq (i,j)$ 且 (x,y) 落在形狀為 λ 的表上。
  Recursion:
  (i) $f^{(0,\cdots,0)} = 1$
  (ii) $f^{(\lambda_1,\cdots,\lambda_m)} = \sum_{k=1}^{m} f^{(\lambda_1,\cdots,\lambda_{k-1},\lambda_k-1,\lambda_{k+1},\cdots,\lambda_m)}$

6.2 Prime

不知道要做啥但放一下

Prime	Root	Prime	Root
7681	17	167772161	3
12289	11	104857601	3
40961	3	985661441	3
65537	3	998244353	3
786433	10	1107296257	10
5767169	3	2013265921	31
7340033	3	2810183681	11
23068673	3	2885681153	3
469762049	3	605028353	3
924844033	5	0	0